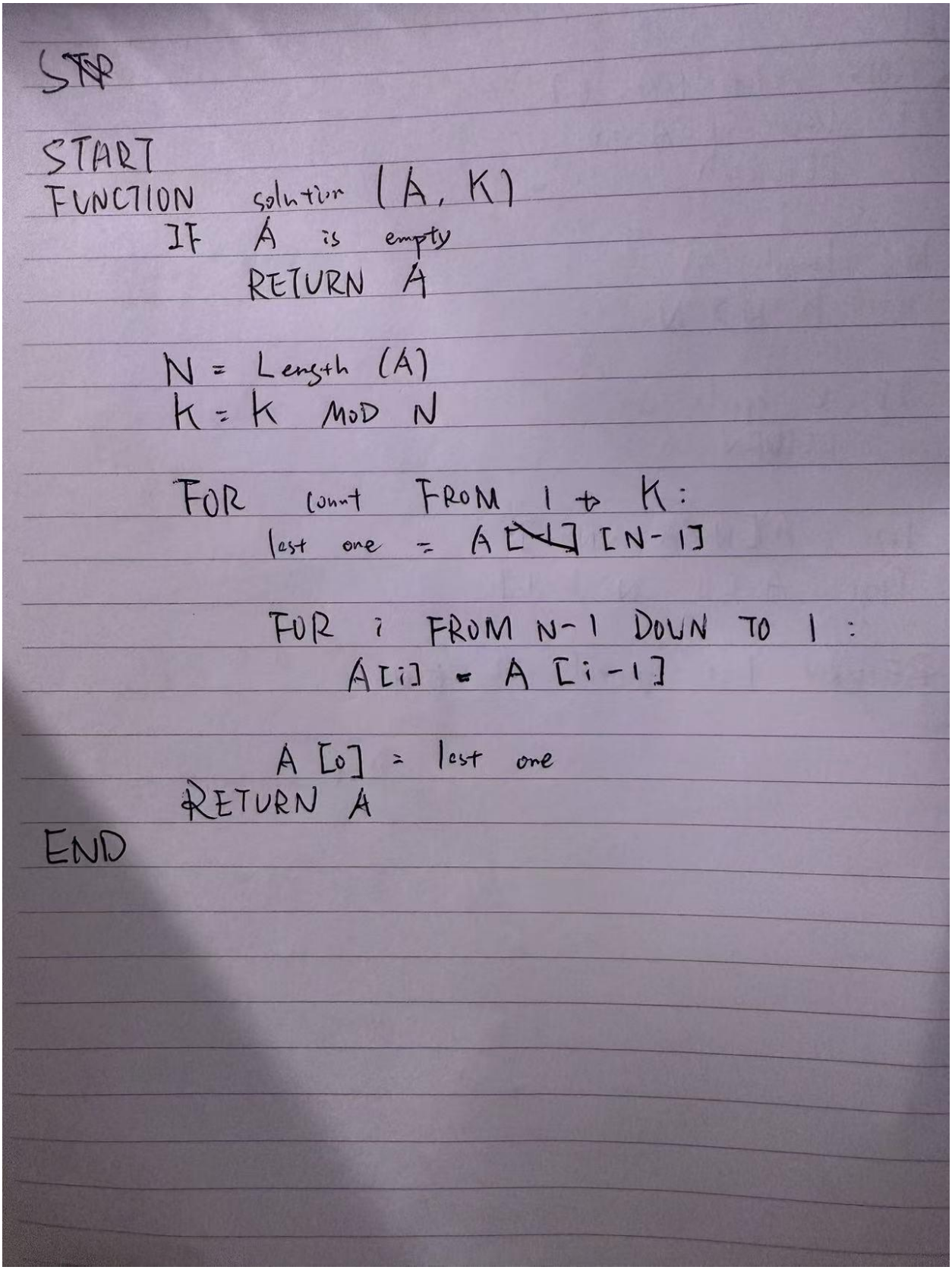
1. CyclicRotation



```
START
FUNCTION solution (A, K)
    IF A is empty
        RETURN A

    N = Length (A)
    K = K MOD N

    FOR count FROM 1 to K:
        last one = A[ ] [N-1]

        FOR i FROM N-1 DOWN TO 1 :
            A[i] = A [i-1]

        A [0] = last one
    RETURN A
END
```

1. CyclicRotation

I want to start a for loop to repeat each step (put out the last element and move it to the front). And considering it could have the empty array, and K is greater than the length of the array. So, need to add more condition on that:

Start by defining the solution and creating a condition for returning the original array when it is empty. Because there is nothing to switch.

For K is greater than array. When each K = len(A) had been done. The array goes back to itself. So only execute the meaningful step is the remainder that K divides by len(A).

Under the for loop, each step we must pick the last element out and move other element goes back one position. Then add the last element to the front.

Finaly return the solution

```
def solution (A, K):

    if not A:  #return the original if array is empty
        return A

    n = len(A)

    steps = K % n #if times K is greater than the array, just have to switch K % n times

    for _ in range(steps):  #pick up the last element each time

        last_element = A[-1]
```

1. CyclicRotation

```
    for i in range(n - 1, 0, -1):  #move every element back by one position except the last one
        A[i] = A[i - 1]


    A[0] = last_element #set last element to the first one

    return A

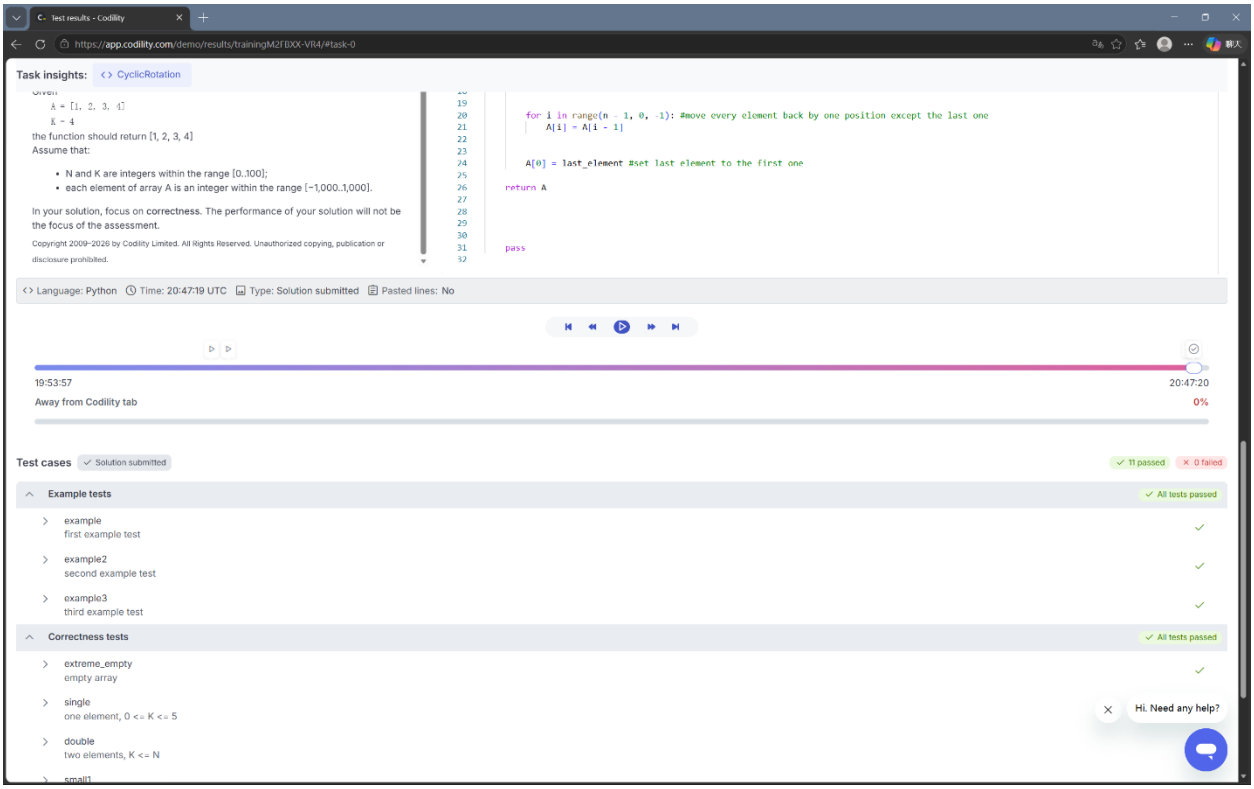

    pass
```

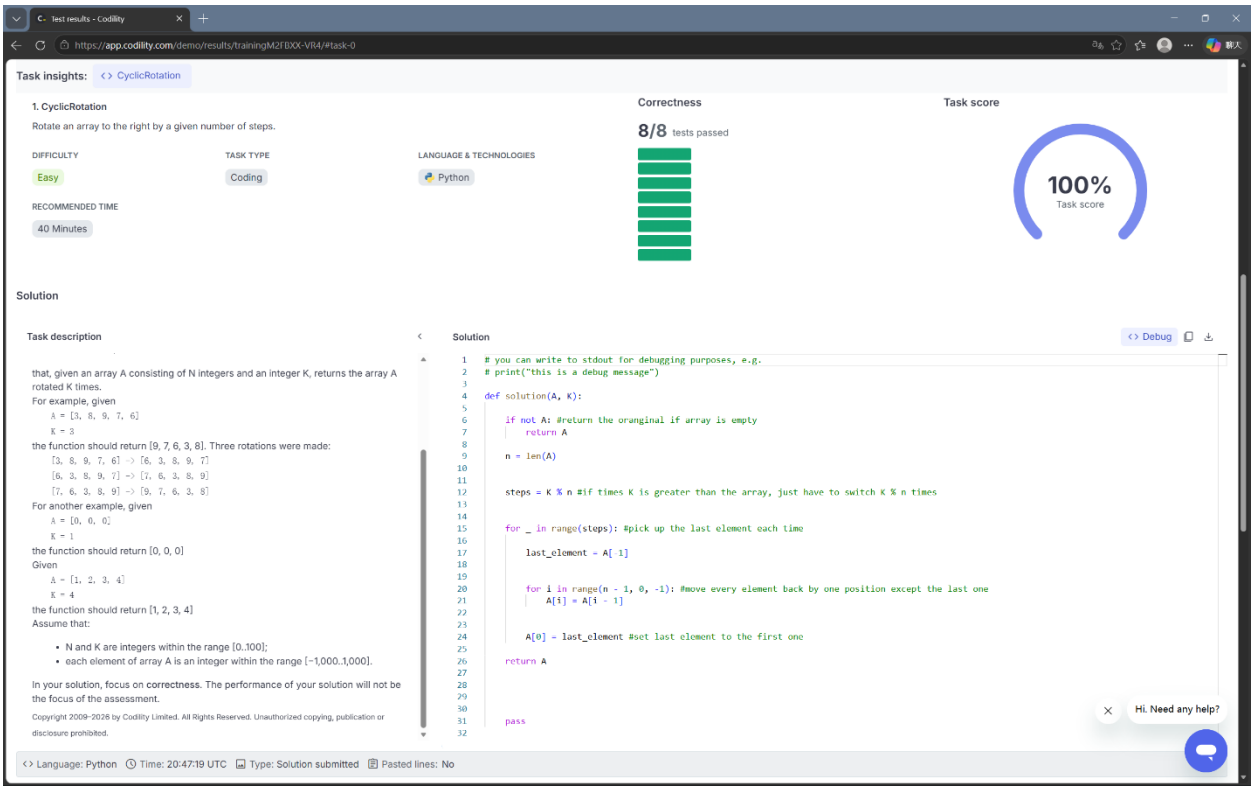
AI Assistance & Citation for Task 1 (Cyclic Rotation):

I utilized ChatGPT to review my initial logic and correct errors to ensure it passes Codility's correctness tests. The AI helped identify edge cases

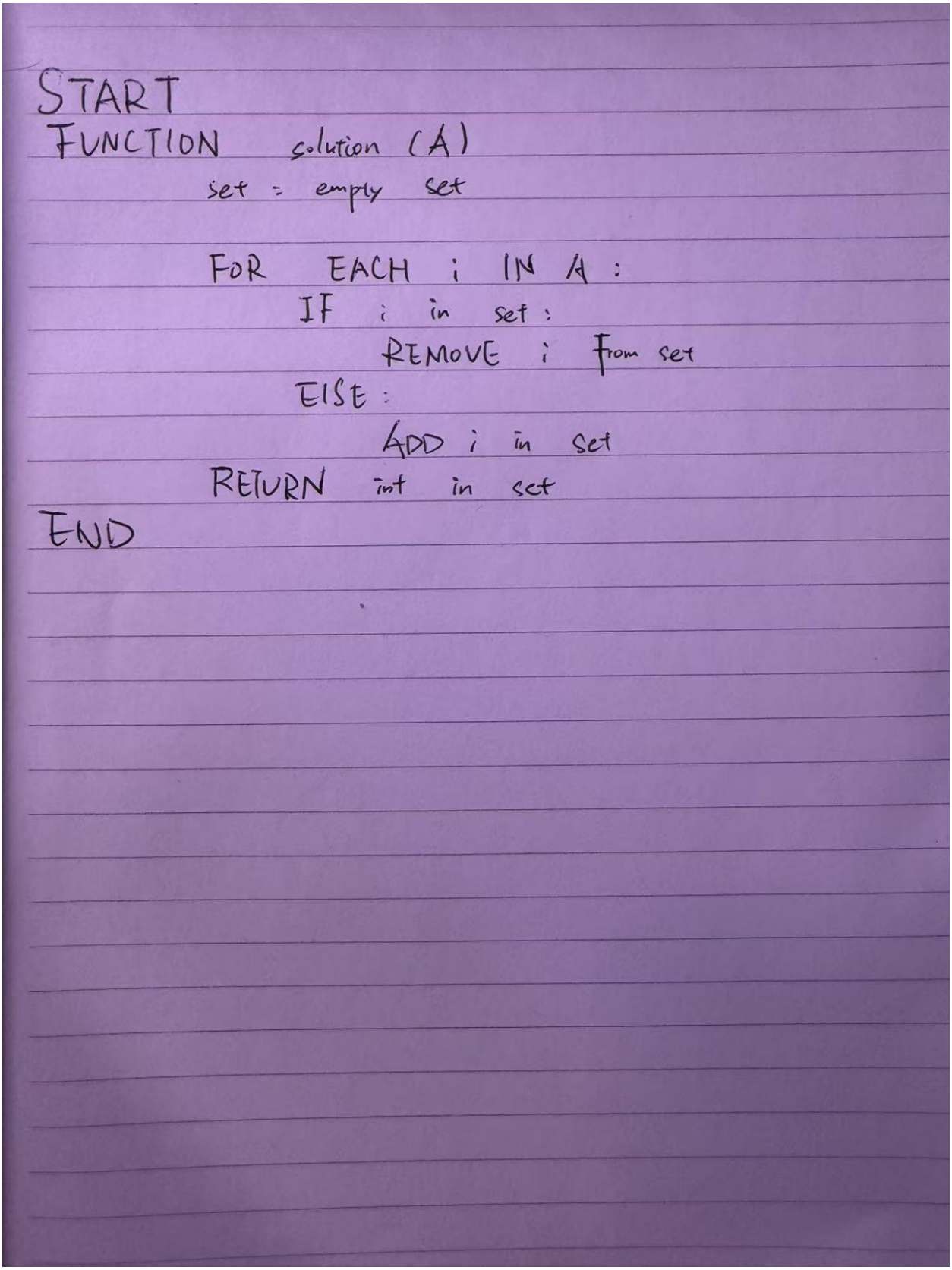
https://chatgpt.com/share/6ae5340f-cca0-83e9-8cdc-a8ea9e1d9857

1. CyclicRotation





2. OddOccurrencesInArray



```
START
FUNCTION solution (A)
    set = empty set

    FOR EACH i IN A :
        IF i in set :
            REMOVE i From set
        EISE :
            ADD i in set
    RETURN int in set
END
```

2. OddOccurrencesInArray

We want to find one alone that does not have the same value element matches no. I am thinking about using scanning each one in the list and when finding one that already has the same value element, then remove that one. If not, add that one. So, after running all the elements in the list. That one alone should be left.

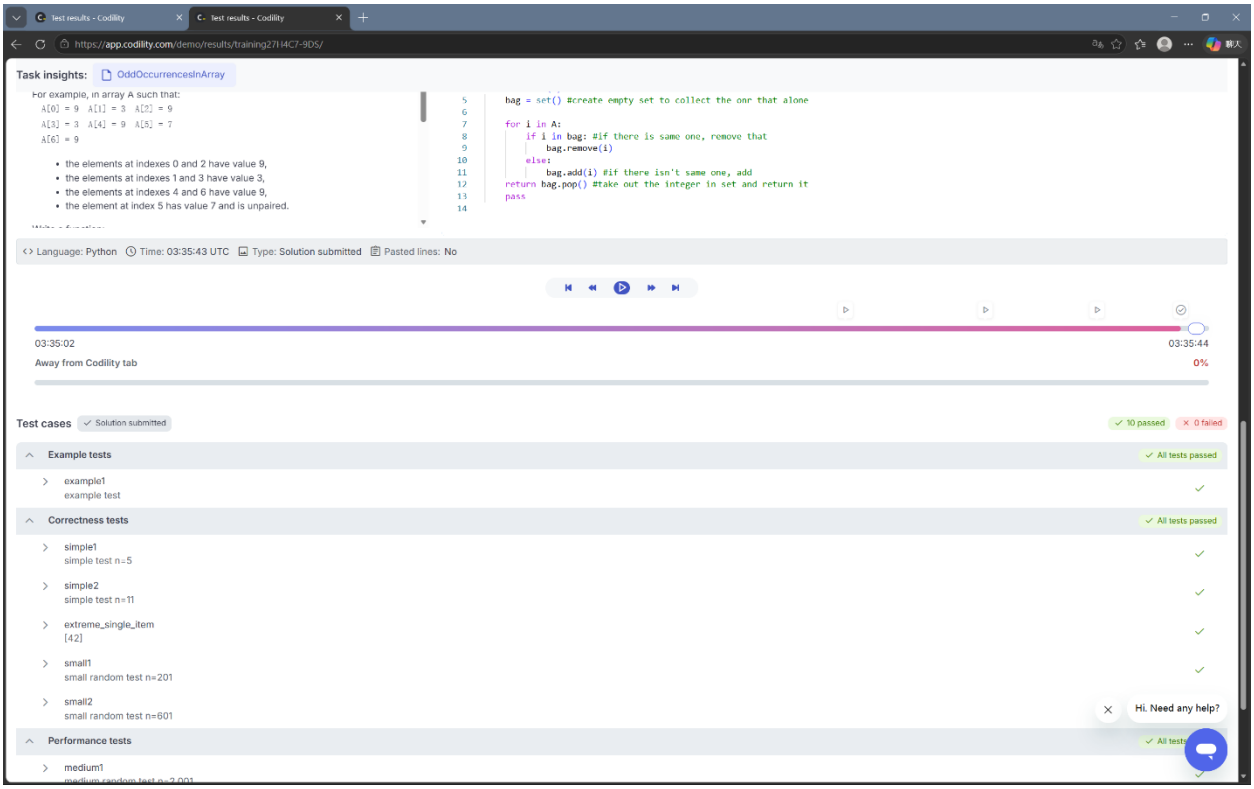
```
def solution(A):
    bag = set() #create empty set to collect the onr that alone

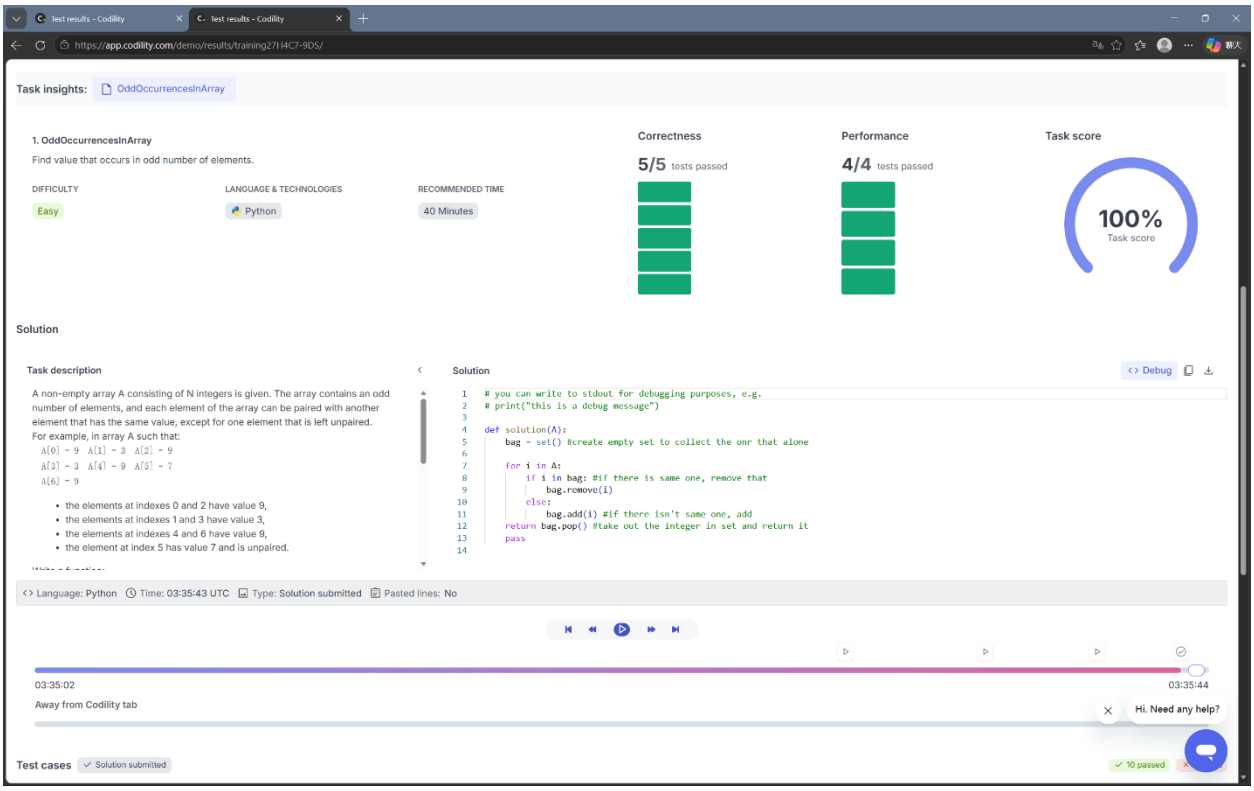

    for i in A:
        if i in bag: #if there is same one, remove that
            bag.remove(i)
        else:
            bag.add(i) #if there isn't same one, add
    return bag.pop() #take out the integer in set and return it
    pass
```

AI Assistance & Citation for Task 1 (Cyclic Rotation):
I utilized ChatGPT to review my initial logic and correct errors to ensure it passes Codility's correctness tests. The AI helped identify edge cases

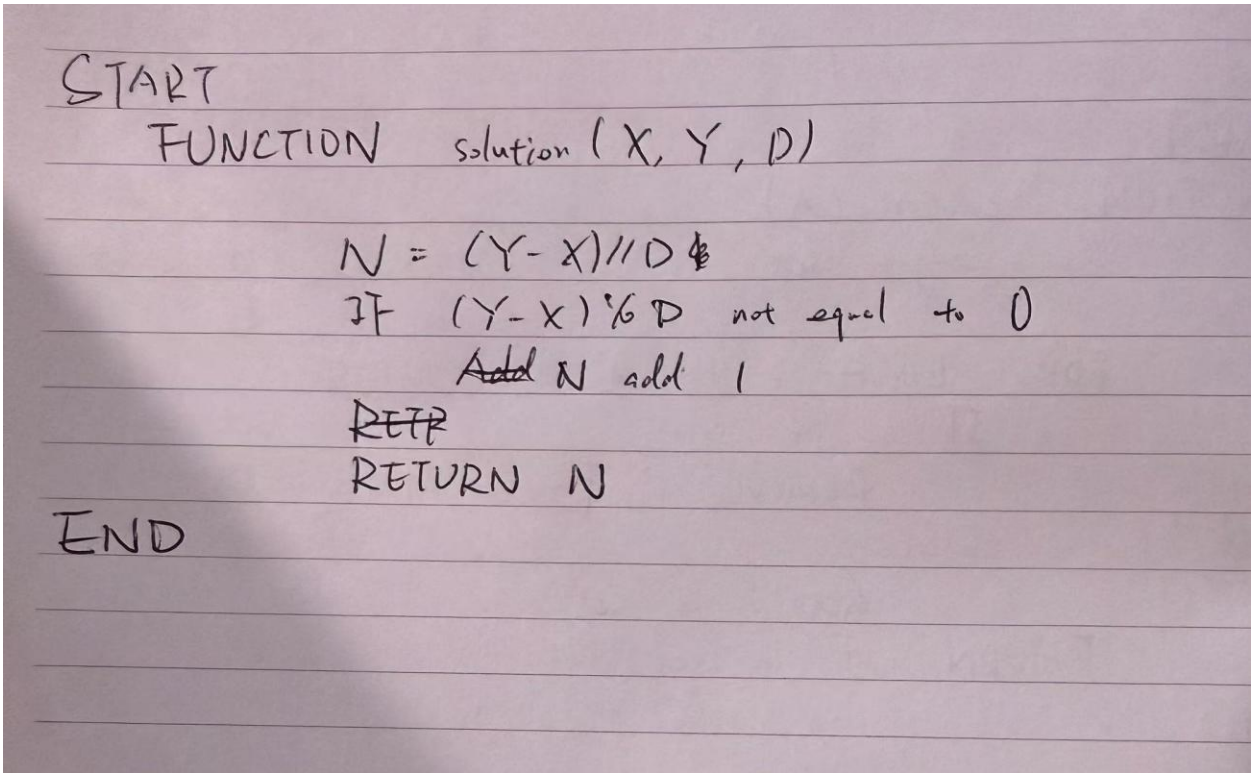
https://chatgpt.com/share/6ae54d2a-be7c-83ea-a5e1-1dd66c71f456

2. OddOccurrencesInArray





3. FrogJmp



We want to calculate the minimum step to getting to place Y. because the step must always be integer. So, I do like to use the distance (Y - X) divided by length of each step. If there is still a little distance that less than one step. I must add one more because can not go half step or anything like decimal.

3. FrogJmp

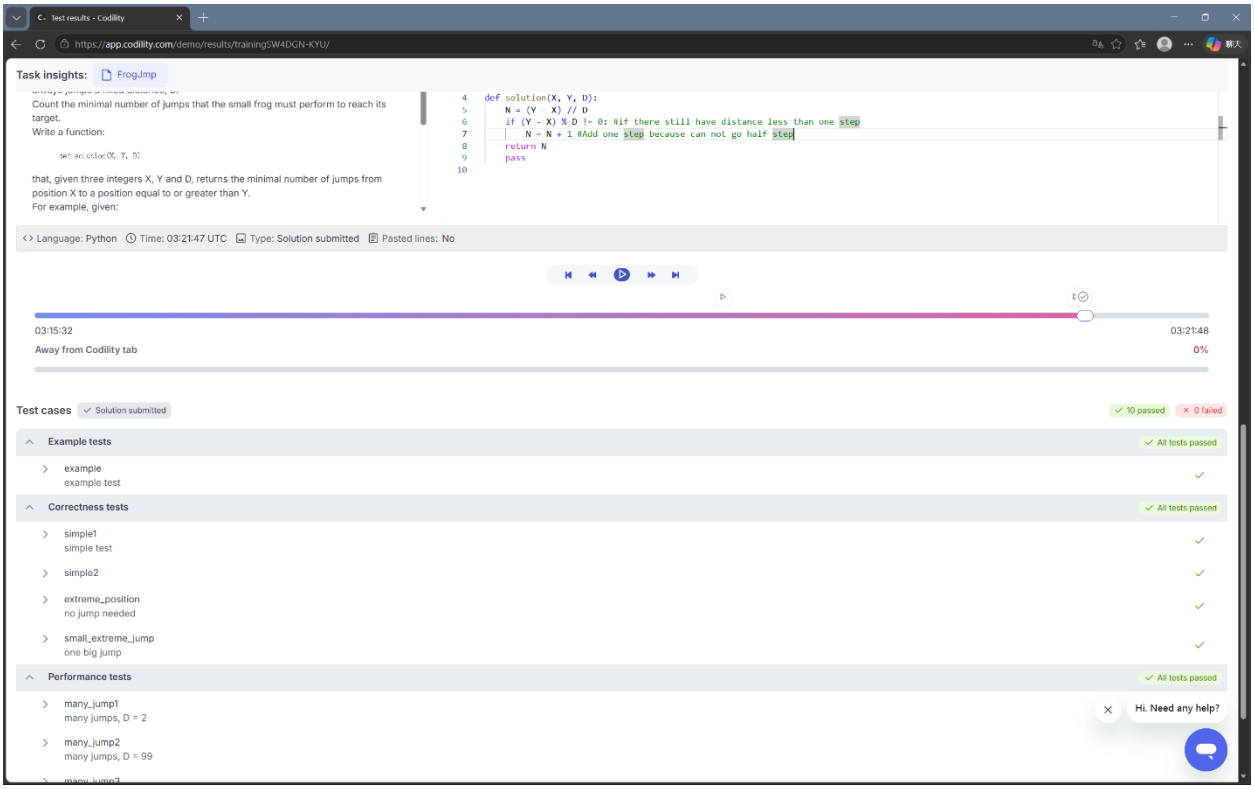
```
def solution(X, Y, D):
    N = (Y - X) // D
    if (Y - X) % D != 0: #if there still have distance less than one step
        N = N + 1 #Add one step because can not go half step
    return N
    pass
```

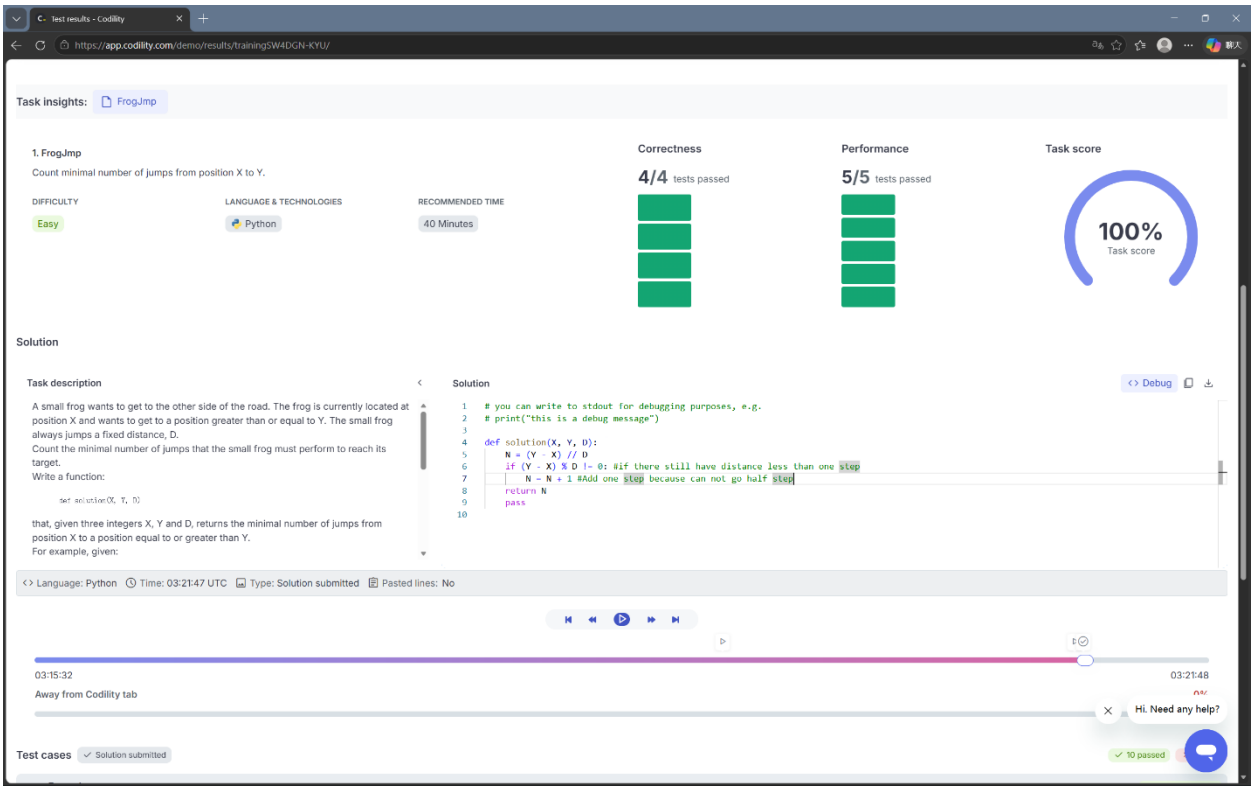
AI Assistance & Citation for Task 1 (Cyclic Rotation):

I utilized ChatGPT to review my initial logic and correct errors to ensure it passes Codility's correctness tests. The AI helped identify edge cases

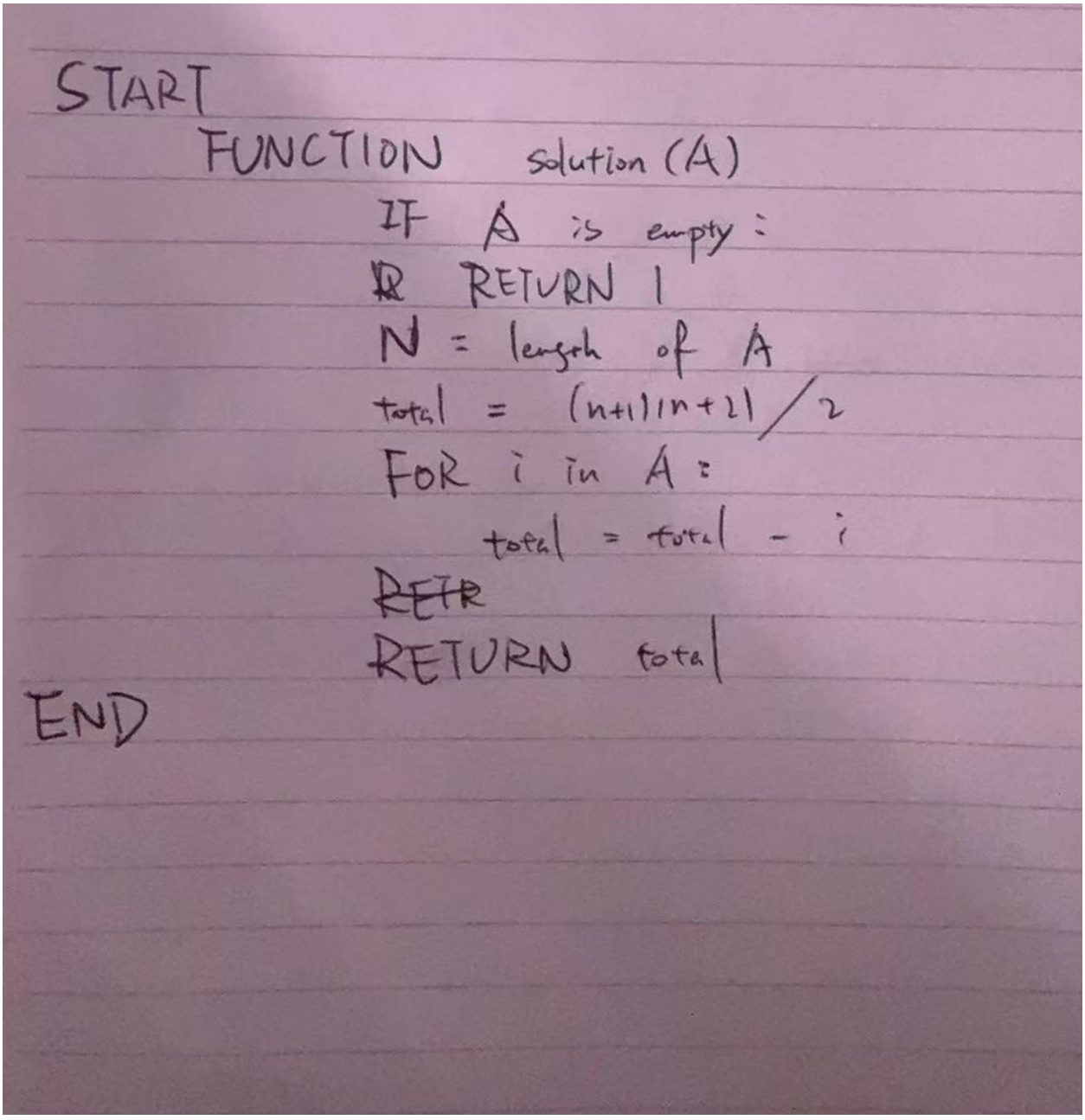
https://chatgpt.com/share/6ae560a7-0218-83ea-bf8a-bc659ef20d1f

3. FrogJmp


Test results - Codility
https://app.codility.com/demo/results/trainingSW4DGN-KYU/
Task insights:
FrogJmp
Count the minimal number of jumps that the small frog must perform to reach its target.
Write a function:
that, given three integers X, Y and D, returns the minimal number of jumps from position X to a position equal to or greater than Y.
For example, given:
def solution(X, Y, D):
N = (Y - X) // D
if (Y - X) % D != 0: #if there still have distance less than one step
N = N + 1 #Add one step because can not go half step
return N
pass
Language: Python
Time: 03:21:47 UTC
Type: Solution submitted
Pasted lines: No
03:15:32
03:21:48
Away from Codility tab
0%
Test cases
Solution submitted
10 passed
0 failed
Example tests
All tests passed
example
example test
Correctness tests
All tests passed
simple1
simple test
simple2
extreme_position
no jump needed
small_extreme_jump
one big jump
Performance tests
All tests passed
many_jump1
many jumps, D = 2
many_jump2
many jumps, D = 99
Hi. Need any help?


Test results - Codility
https://app.codility.com/demo/results/trainingSW4DGN-KYU/
Task insights:
FrogJmp
1. FrogJmp
Count minimal number of jumps from position X to Y.
DIFFICULTY
Easy
LANGUAGE & TECHNOLOGIES
Python
RECOMMENDED TIME
40 Minutes
Correctness
4/4 tests passed
Performance
5/5 tests passed
Task score
100%
Task score
Solution
Task description
A small frog wants to get to the other side of the road. The frog is currently located at position X and wants to get to a position greater than or equal to Y. The small frog always jumps a fixed distance, D.
Count the minimal number of jumps that the small frog must perform to reach its target.
Write a function:
that, given three integers X, Y and D, returns the minimal number of jumps from position X to a position equal to or greater than Y.
For example, given:
Solution
Debug
you can write to stdout for debugging purposes, e.g.
print("this is a debug message")
def solution(X, Y, D):
N = (Y - X) // D
if (Y - X) % D != 0: #if there still have distance less than one step
N = N + 1 #Add one step because can not go half step
return N
pass
Language: Python
Time: 03:21:47 UTC
Type: Solution submitted
Pasted lines: No
03:15:32
03:21:48
Away from Codility tab
Hi. Need any help?
Test cases
Solution submitted
10 passed

4. PermMissingElem



```
START
    FUNCTION solution (A)
        IF A is empty :
        RETURN 1
        N = length of A
        total = (n+1)(n+2) / 2
        FOR i in A :
            total = total - i
        RETR
        RETURN total
END
```

4. PermMissingElem

We want to get the integer that missed. So, I am thinking about adding them up and getting the total value of the number in list. And used the math formular n(n+1)/2 to get the total value that if nothing missed.

Then total value that nothing was missing, minus the total value in the list. Can get the value of integer that we missed.

But considering efficiency, it can be used For loop to minus the integer one by one, the answer will be the same.

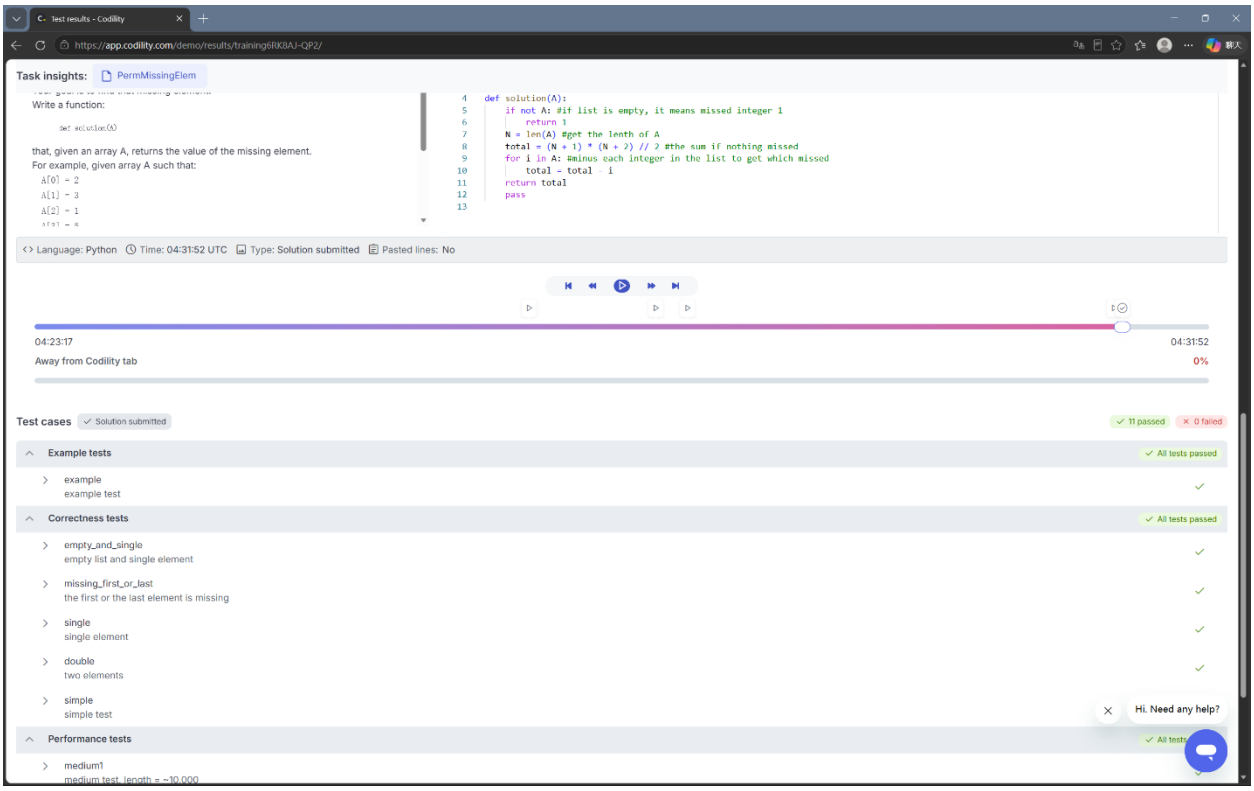
But in this case, N is N + 1. So, we must do a little change to the formular and get (n+1) (n+2)/2
If the list is empty, it means it should have integer 1 in the list, but it missed. Caused by an empty list.

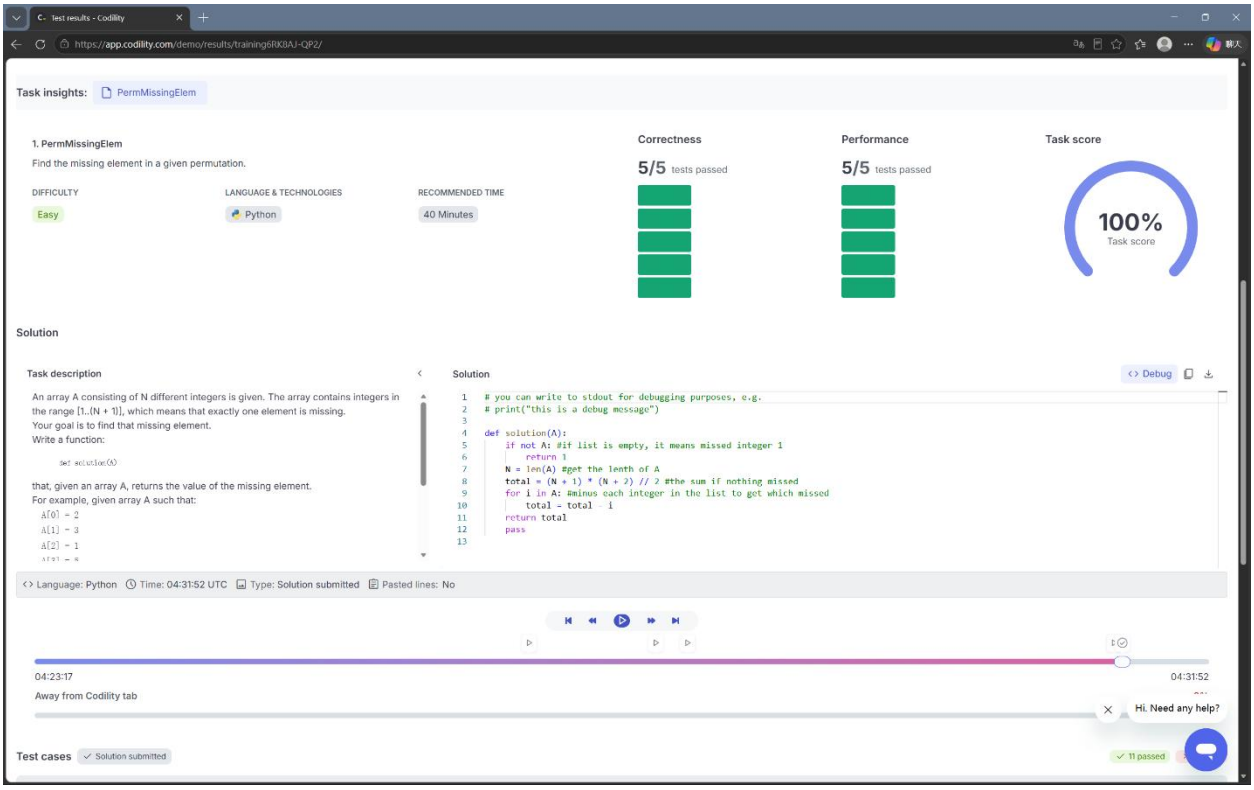
```
def solution(A):
    if not A: #if list is empty, it means missed integer 1
        return 1
    N = len(A) #get the lenth of A
    total = (N + 1) * (N + 2) // 2 #the sum if nothing missed
    for i in A: #minus each integer in the list to get which missed
        total = total - i
    return total
```

AI Assistance & Citation for Task 1 (Cyclic Rotation):
I utilized ChatGPT to review my initial logic and correct errors to ensure it passes Codility's correctness tests. The AI helped identify edge cases

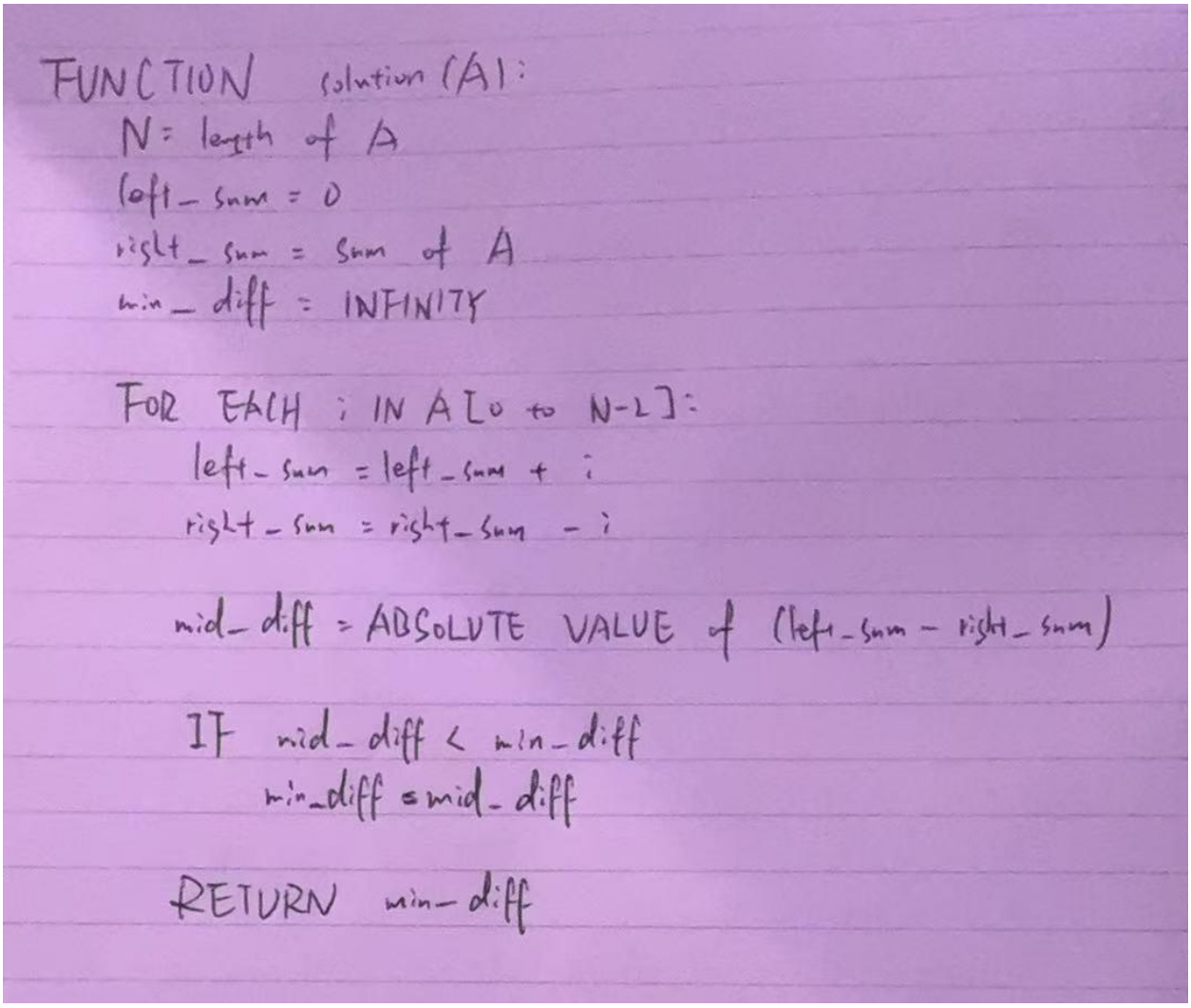
https://chatgpt.com/share/6ae5753d-6328-83ea-8b16-d08a18d03429

4. PermMissingElem





5. TapeEquilibrium



We want to get the smallest absolute difference between two parts. I am first thinking about moving all of them into one side and picking each to the other side and calculating the absolute difference at each step.
if the absolute difference is getting smaller, it means we get the right step, so we keep this answer and keep going, until the absolute difference stop to get smaller, that means there is minimum absolute difference we can get.
we do not pick up the last one in the right list.

5. TapeEquilibrium

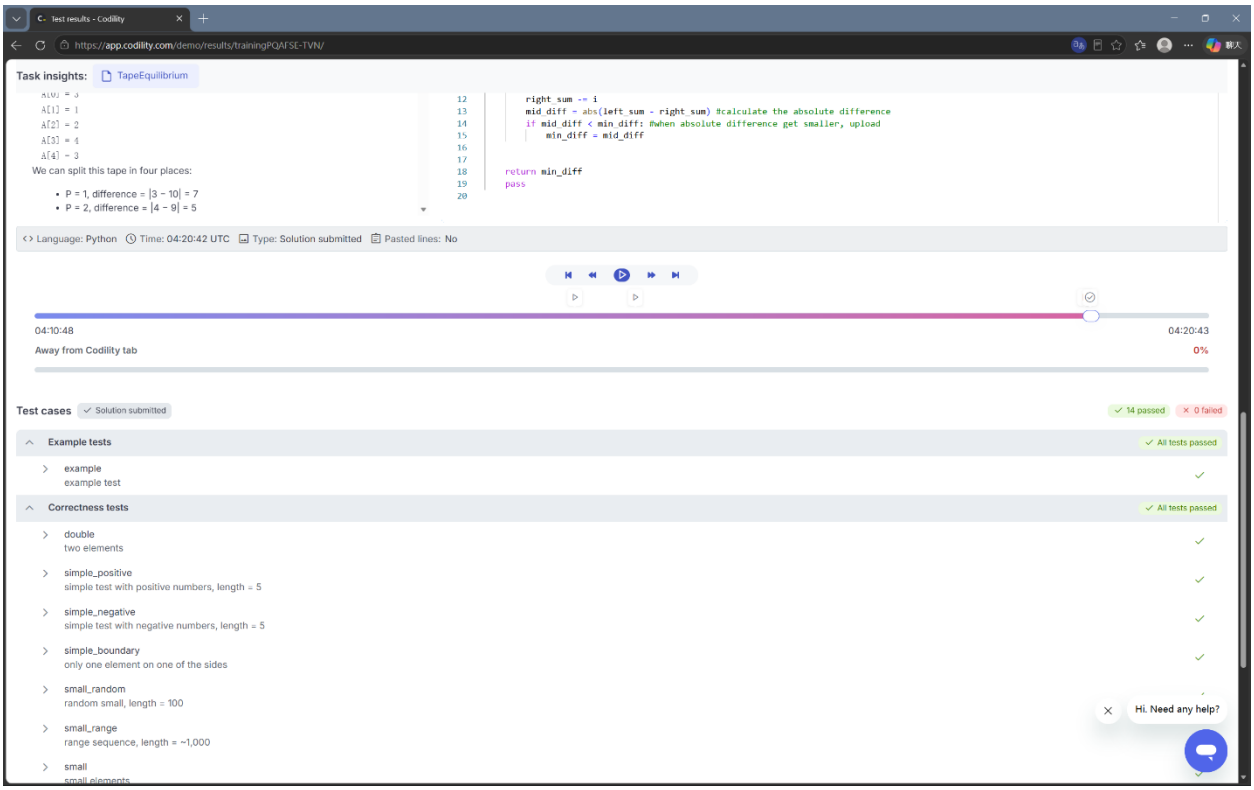
AI Assistance & Citation for Task 1 (Cyclic Rotation):
I utilized ChatGPT to review my initial logic and correct errors to ensure it passes Codility's correctness tests. The AI helped identify edge cases
https://chatgpt.com/share/6ae6c400-9170-83ea-9f4e-c159925b304e

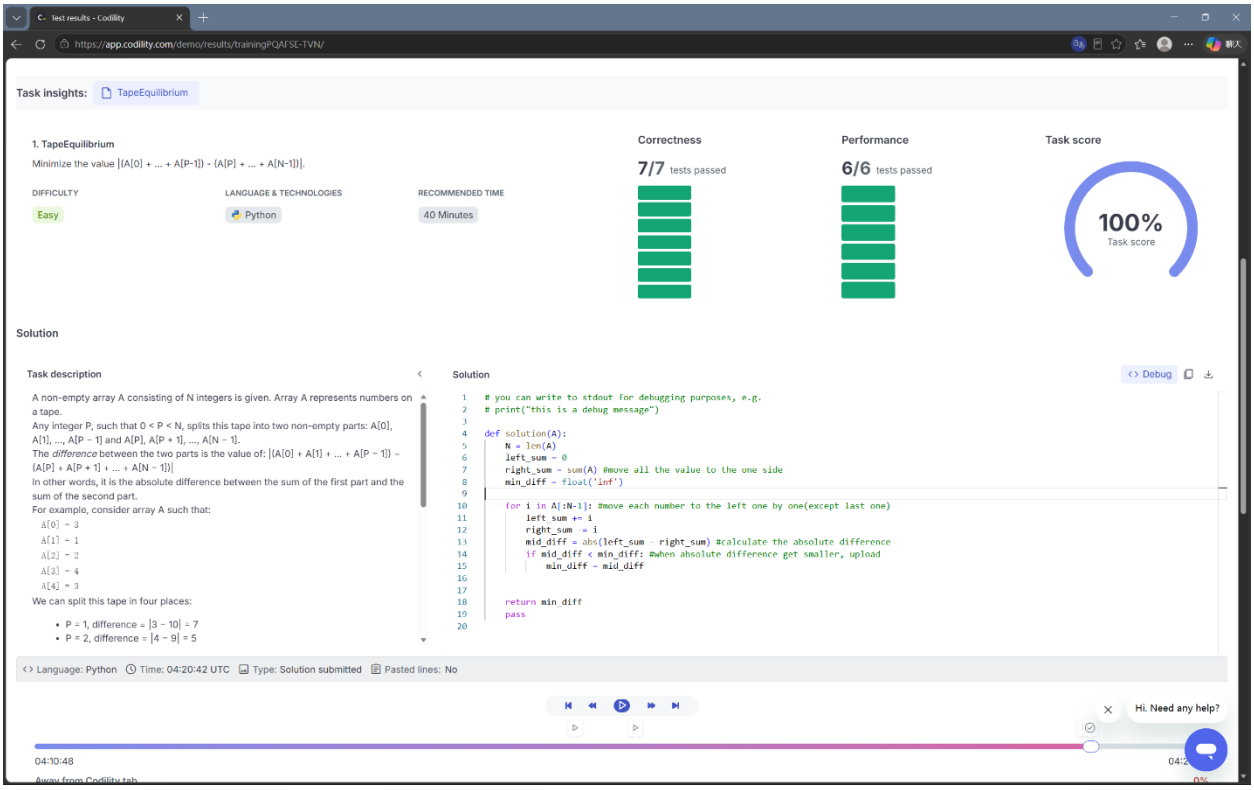
```
def solution(A):
    N = len(A)
    left_sum = 0
    right_sum = sum(A) #move all the value to the one side
    min_diff = float('inf')

    for i in A[:N-1]: #move each number to the left one by one(except last one)
        left_sum += i
        right_sum -= i
        mid_diff = abs(left_sum - right_sum) #calculate the absolute difference
        if mid_diff < min_diff: #when absolute difference get smaller, upload
            min_diff = mid_diff

    return min_diff
```

5. TapeEquilibrium





GitHub Link

Commits · lyz20060803-boop/Assignment1_Codility